AI Concepts: Tokens and Embeddings

Beginner-friendly notes with examples for Spring AI and Ollama.

1. Tokens

A token is a piece of text that a language model processes. Depending on the tokenizer, a token may be a whole word, part of a word, punctuation, or another text unit. Tokens are not the same as words, and token boundaries depend on the model.

Example

Input sentence: "Hello, how are you?"

A tokenizer might split this sentence into units resembling Hello | , | how | are | you | ?. This is illustrative only; actual tokenization varies by model.

Common token terms

- Input tokens: tokens in the prompt sent to the model, including system instructions, user text, and supplied context.
- Output tokens: tokens generated by the model in its response.
- Context window: the maximum amount of input and output content a model can handle in a request, measured in tokens.
- Cost and performance: for hosted APIs, token counts can affect cost; token volume can also affect latency and context usage.

2. Embeddings

An embedding is a numerical vector produced by an embedding model from text (or other supported data). The vector captures learned patterns useful for comparing the text with other text. The individual numbers usually do not have simple, human-readable meanings.

Example

Input text: "I forgot my password"

With the configured nomic-embed-text model in the example application, the output vector has 768 dimensions. The vector may begin with values such as [0.0462, -0.0042, -0.1223, 0.0216, ...]. These sample values are from the example discussed; the full vector contains 768 values.

What does 768 dimensions mean?

It means the vector contains 768 numerical coordinates. It does NOT mean there are 768 separate meanings, 768 words, or one manually assigned concept per number. The dimensions work together as one learned representation.

For the same embedding model, a different sentence such as 'I forgot my password for this account' will generally produce different values but the same vector length (768 for this model/configuration). Vector length is determined by the embedding model, not by the number of words in the sentence.

3. Tokens vs. Embeddings

Aspect	Tokens	Embeddings
What it is	Text units used by the model	A vector of numerical values
Example	"I forgot my password" is split into tokenizer units	A 768-number vector from nomic-embed-text
Main purpose	Process text and measure text length/context	Represent text for similarity search and retrieval
Human interpretation	Tokens can often be viewed as text pieces	Individual dimensions are generally not directly interpretable
Used in RAG	The LLM reads prompt/context tokens	Embeddings help retrieve relevant document chunks

4. How embeddings support semantic search and RAG

1. Split knowledge documents into manageable chunks.
2. Generate an embedding vector for each chunk.
3. Store vectors with the text and metadata in a vector database.
4. Generate an embedding for the user's question.
5. Compare the query vector with stored vectors and retrieve the most relevant chunks.
6. Give the retrieved text to the LLM so it can generate a grounded answer.

Example: a question 'I cannot log in' may retrieve a document titled 'Reset your password' because their embeddings may be semantically similar even though the wording differs.

5. Spring AI + Ollama example

Example application.yml configuration:

```
spring:
  ai:
    ollama:
      base-url: http://localhost:11434
      embedding:
        options:
          model: nomic-embed-text
```

Download the model:

```
ollama pull nomic-embed-text
```

Generate a vector in Java:

```
float[] vector = embeddingModel.embed("I forgot my password");
System.out.println(vector.length); // 768 for this model/configuration
```

6. Key points to remember

- Tokens are pieces of text; they are not always whole words.
- Embeddings are vectors of numbers representing learned patterns in text.
- 768 dimensions means 768 coordinates in one vector, not 768 separate meanings.
- Changing the sentence generally changes the vector values, while the configured model's vector dimension stays fixed.
- Embeddings enable similarity search; a vector database stores/searches vectors; RAG retrieves relevant content; an LLM generates the answer.

An embedding helps you find text with similar meaning; it is not a replacement for storing the original text.